Before you start

Each of these six projects maps to one skill an AI engineering interview will test. Built together they cover the full surface: retrieval, agents, operations, evaluation, model economics, and safety. Built well, any one of them carries a whole interview loop on its own.

Read this page first, because it decides whether the next forty pages are worth your time.

Sequence them, do not sprint them

Build in the order they appear. Project 1 gives you a system worth evaluating. Project 4 evaluates it. Project 6 attacks it. Project 5 makes it cheaper. That dependency chain is deliberate, and a portfolio where the projects reference each other reads as an engineer with a system rather than a student with a checklist.

If you only have time for two, build Project 1 and Project 3. Retrieval plus infrastructure is the most common shape of a real AI engineering job.

What makes a project count

- A number. Every one of these projects produces a measured result: an accuracy delta, an availability figure, an agreement score, a break even volume. A project with no number is a tutorial you followed.
- A README that opens with the result. Benchmark table or demo video first, architecture second, setup instructions last. Nobody scrolls.
- A ninety second demo video. Most reviewers will never run your code. They will watch the video, read the top of the README, and decide.
- Honest failure notes. A section on what did not work and why is the single strongest credibility signal in a portfolio repository.
- Real data. Public filings, your own documents, open datasets. Toy data produces toy results and interviewers can tell instantly.

How each project is laid out

Why it signals. What this project proves that a tutorial does not, and the interview question it answers for you.

Five phases. The build, in order, with the specific decisions and the places people get stuck.

Done when. The finish line, so you stop building and start shipping.

Resume line and mistakes. How to describe it, and the four ways these projects usually go wrong.

Estimates assume ten to twelve hours a week. Halve them if this is full time work, and do not be discouraged if a phase takes twice as long as the estimate. Phase 1 always does.

PROJECT 1 · RAG

Knowledge Graph RAG for Enterprise Data

Python · Neo4j · LangChain · Claude API · pgvector · FastAPI

Estimated build time: 3 to 4 weeks part time

Why this project signals

Almost every candidate has built vector RAG. Very few have built retrieval that answers questions requiring two or three hops across related entities. This project proves you understand where embeddings fail and what to do about it, which is exactly the follow up interviewers use to separate people who read about RAG from people who have debugged it.

PHASE 1

Extract entities and relationships into Neo4j

- Choose a corpus that actually has relationships. SEC filings, an internal wiki, research papers with citations, or product docs with dependency chains all work. A folder of unrelated blog posts has nothing to extract and the project dies here.
- Constrain the ontology before you write any code. Define five to ten entity types and eight to fifteen relationship types. An open ended extract all entities prompt produces a graph nobody can query.
- Chunk the documents, then run an extraction pass with Claude returning a strict schema: entity type, canonical name, relationship type, source chunk id, and confidence. Validate every response against the schema and retry the failures.
- Entity resolution is the hard part and the part that gets skipped. Acme Corp, Acme Corporation, and ACME must collapse into one node. Normalize, compare with embedding similarity above a tuned threshold, and store an alias list on the node.
- Write with MERGE rather than CREATE so re running ingestion is idempotent. Every edge carries source chunk id so you can cite the sentence that justified it.
- Budget the extraction cost per document before you run the full corpus, and cache results by document hash. This is where people accidentally spend two hundred dollars in an afternoon.

PHASE 2

Build the vector index alongside the graph

- Embed the same chunks into pgvector with metadata: document id, section path, date, and the entity ids that chunk mentions.
- Both stores share the same chunk ids. That shared key is the whole trick. It lets you cross from a graph path back to the original text, and from a retrieved passage into the graph neighborhood around it.
- Index with HNSW, tune ef_search, and measure recall at k on a small labeled set before moving on. Do not build the router on top of retrieval you have not measured.

PHASE 3

Route questions to the right retrieval path

- Write a cheap router: a small model call with a few shot prompt returning an enum, plus a low confidence fallback that runs both paths and merges.
- Route to the graph for connection questions, multi hop chains, comparisons across entities, and aggregations over relationships. Route to vectors for definitions, policy lookups, and single fact questions.
- On the graph path, extract entities from the question, resolve them to node ids, then run a parameterized Cypher query. Never let the model emit raw Cypher against your database. Keep a template library keyed by query type and let the model fill parameters only. This is a security control and it also makes results reproducible.
- Log every routing decision with the question and the outcome. You need this data for Phase 5 and you will not be able to reconstruct it later.

PHASE 4

Merge both sources into one grounded answer

- Graph traversal returns paths and vector search returns passages. Convert paths into readable statements before they reach the prompt, because raw triples generate awkward text.
- Deduplicate across the two sets, then assemble context with explicit labels separating graph derived facts from retrieved passages.
- Require a citation per claim, then validate that every citation resolves to a chunk id that was actually retrieved. Reject and regenerate when it does not. This costs nothing and eliminates invented sources.

PHASE 5

Benchmark against plain vector RAG and publish the delta

- Build a question set of fifty to a hundred items, stratified by difficulty: single hop, two hop, three hop, aggregation, and out of scope questions the system should refuse.
- Run both systems on the same set and report accuracy broken out by hop count. The expected shape is parity on single hop questions and a widening gap as hops increase. That curve is the story.
- Report latency and cost per query for both, plus the one time ingestion cost of the graph. Being honest that graph RAG is slower and more expensive to build is what makes the accuracy claim credible.
- This benchmark table is the portfolio artifact. Put it at the top of the README, above the architecture diagram.

Done when

A FastAPI endpoint answers questions with validated citations, and the README opens with a benchmark table comparing your system to a vector only baseline by hop count.

> **Resume line:** Built a hybrid knowledge graph and vector RAG system over N documents; raised